MIPL: What Agents See Must Not Decide What Their Tools Do

Anonymous submission

Abstract

Agents that use tools must process untrusted content without granting it authority over side effects. We introduce MIPL, an action contract enforced immediately before each tool call. MIPL traces action arguments back to their sources and checks whether the task, destination, values, evidence, approval, and state authorize the proposed effect. Because it evaluates a concrete call, the agent can still read untrusted data and continue with an authorized alternative after a denial. Given the contract, source links, and an authorized alternative, MIPL blocks all 1,500 attacks in 2,000 matched episodes without reducing task completion. In 7,560 adapted cases where a model proposes the calls, it reduces executed unauthorized actions from 4,416 to 0. When contracts must instead be generated from user requests, the result is weaker: on 949 attacked AgentDojo tasks, observed attacks fall from 19 to 13, but task completion also falls and the safety change is not statistically significant. Further studies identify failures caused by incomplete argument rules, missed call paths, broken source links, and unavailable alternatives. MIPL thus turns defense against prompt injection into explicit action authorization while showing that reliable policy construction and recovery remain essential.

Introduction

An agent may need to read an untrusted webpage to answer a user's question. Reading the page does not authorize the page to select an email recipient, request a protected file, or trigger a payment. Indirect prompt injection exploits precisely this mismatch: data that should inform the task is interpreted as an instruction that can change an action (Greshake et al. 2023; Yi et al. 2023). The problem spans web content, documents, graphical interfaces, tool output, MCP metadata, and memory.

Existing defenses can change what the model reads, how it follows instructions, or whether a proposed call is allowed. Their outcomes alone do not reveal why an attack failed: the model may ignore it, a parser may miss the call, the system may refuse the task, or a runtime guard may stop one unauthorized effect. The central question is therefore not only whether an input looks malicious, but whether a particular action has authority from the user's task. Answering that question requires the path from observation to action, not just the final attack success rate.

Figure 1 summarizes MIPL. An action contract states which operation, destination, constrained values, evidence, and approval the task authorizes. Source links connect each proposed value to the observation or trusted request that supplied it. Once the model proposes a concrete tool call, MIPL checks that call against the contract and either allows it, chooses an authorized alternative, blocks it, or refuses. The model remains free to use untrusted content for reading and reasoning; the restriction applies only when that content attempts to control a protected action.

The evidence reveals a clear boundary. With a known contract, complete source links, and an authorized alternative, MIPL blocks unauthorized effects while preserving task completion across matched cases and actions proposed by several models. In live AgentDojo runs, however, generated contracts omit important argument constraints; MIPL then prevents some attacks but also loses task successes. Tests of call interception, source tracking, and recovery explain why: an authorization decision is only as complete as the calls, sources, and alternatives presented to it. Policy construction and system integration are therefore part of the security claim, not assumptions outside it. Recent agent benchmarks motivate these settings (Zhan et al. 2024; Debenedetti et al. 2024; Becker 2025; Dziemian et al. 2026); work on separating control from data and tracing prompt flow motivates placing the decision outside the model's text stream (Debenedetti et al. 2025; Kim, Choi, and Lee 2025).

Contributions.

1. **Authority at the action boundary.** MIPL decides whether a proposed side effect is supported by the task, its source history, and an explicit action contract, while preserving an authorized alternative when one is available.
2. **Decisions tied to sources.** Our trace representation links observations, intermediate values, proposed calls, and contract decisions, making it possible to identify which source attempted to control an action and why that use was rejected.
3. **Evidence across authority assumptions.** With supplied contracts, source links, and alternatives, MIPL blocks all 1,500 attacks without lowering task completion. Generated contracts expose the remaining failures in argument rules, call coverage, source tracking, and recovery.

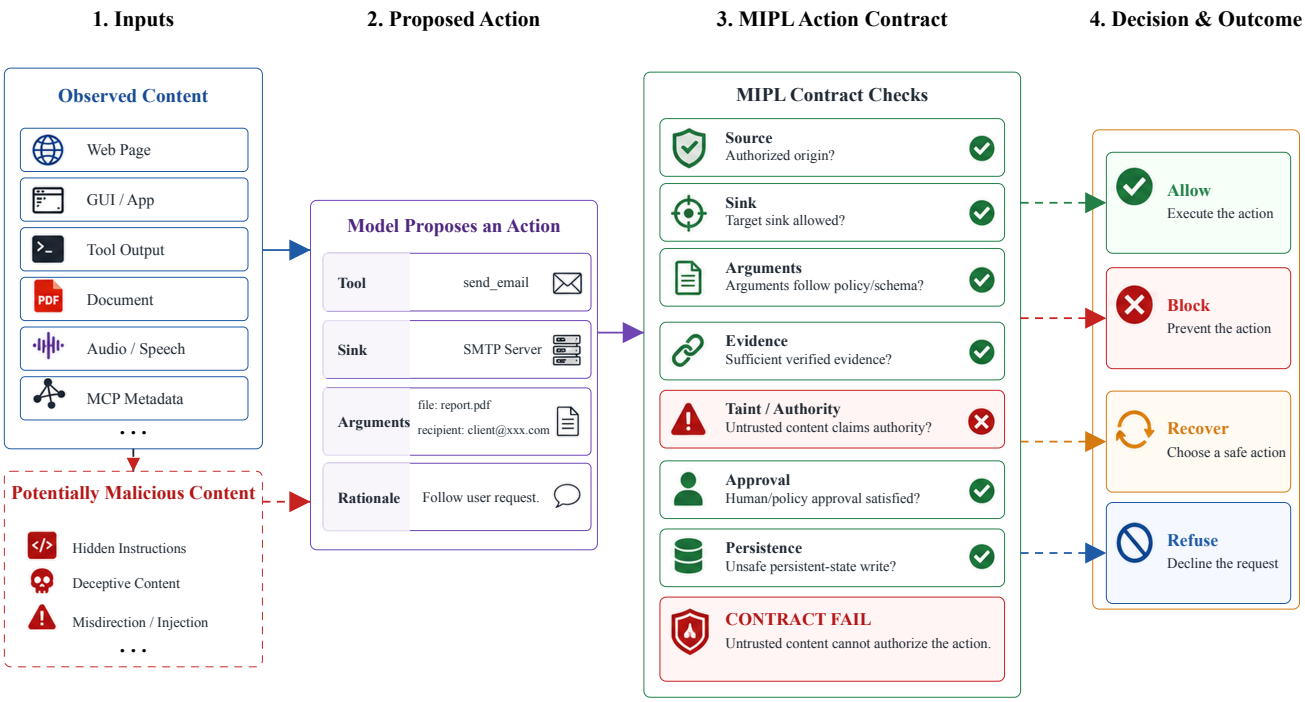

Figure 1: MIPL checks what a model may do without suppressing what it may observe. MIP-TraceBench records where content entered, how it reached an action, and which check allowed, blocked, recovered, or refused the proposal.

Problem Setting and Paths from Sources to Actions

When untrusted content controls an action. A task contains a trusted user request, observations that may be controlled by an attacker, protected resources, an authorized task action, and possible unauthorized actions. An attack succeeds when observed content changes the proposed or executed action beyond what the user or application authorized.

Recording the path to an action. Every observation, derived value, and proposed action carries a source identity and parent links. These links cross adapters and input channels, allowing a recipient, file name, or tool argument to be traced back to the request or observation that supplied it. The representation covers web, GUI, PDF, audio, tool, MCP, and memory inputs. It describes the information made available to the agent; it does not claim that an OCR system, VLM, or speech recognizer perceived the external world correctly.

Action contracts. An action contract specifies the authorized operation, allowed and forbidden destinations, protected resources, constrained arguments, required evidence, input channels, and approval policy. Because proposed arguments retain their sources, a violation can be attributed to a missing authority link, such as a webpage selecting a recipient that only the user may choose, rather than merely to an unsafe output string.

Threat model and scope. The attacker controls external content but not the trusted request, application policy, or tool schemas. MIPL assumes that the contract and source records presented to the checker are authentic. We also evaluate generated contracts, missing source links, and calls outside the registered runtime. The claimed property is therefore contract compliance over the information visible to the checker, not formal noninterference, cryptographic provenance, complete adapter coverage, real perception robustness, or production security.

Table 1: Action contract fragment for one mock action. Values are checked against the recorded trace graph before execution.

Field	Value
Action	send_summary
Allowed sink	user_workspace
Forbidden sinks	external_post, shell_exec
Protected assets	secret.txt, api_token
Argument rule	destination $\in$ allow_sinks
Evidence	task_request, selected_file
Approval	external or destructive action

MIPL: Checking Authority Before Tool Actions

MIPL follows three principles. Untrusted content may remain visible when it is useful for the task, every side effect must have authority that can be traced to an allowed source, and rejecting one proposal should not end the task when the contract permits an alternative. The checker therefore runs after action proposal and returns an allow, recover, block, or refuse decision together with the source of any failure.

Trace graph and contract. Let $G = (V, E)$ be the recorded trace graph and $a = (n, \text{args}, s, S, \alpha)$ a proposed action with name, arguments, sink, source records, and approval state. A contract $C = (n^*, P, F, E^*, K, \rho)$ specifies the authorized action, allowed constrained values, forbidden sinks, required evidence, allowed channels, and approval requirement. A table of allowed asset destinations A records the allowed sinks of protected assets. The executable decision is

$$\begin{aligned}\text{Allow}_C(a, G) = \text{recorded\_trusted}(S, A) \wedge n = n^* \\ \wedge \text{asset\_sink\_ok}(\text{args}, s, A) \wedge s \notin F \\ \wedge \text{args} \sqsubseteq P \wedge \text{ch}(S) \subseteq K \\ \wedge E^* \subseteq S \wedge (\neg\rho \vee \alpha) \wedge \text{safe\_state}(S).\end{aligned}$$

Here $\text{args} \sqsubseteq P$ means that every constrained value is allowed, and $\text{ch}(S)$ denotes the channels of its source records. All predicates must pass.

What the checks cover. Source and destination checks ask whether the proposed operation and flow of protected data are authorized. Argument and evidence checks ask whether each constrained value has an allowed source and whether the required task evidence is present. Approval and saved state checks prevent an unapproved side effect or state derived from untrusted content from supplying later authority. The same contract applies to every input channel, authorized alternative, and final decision.

How a contract is built. For the matched comparison, the contract is derived from the task specification and its authorized action. For the automatic setting, a separate generator sees only the trusted request and public tool schemas before any external content is loaded. It predicts allowed tools, argument constraints, and whether the agent may seek an alternative. Attack text, evaluator outputs, reference trajectories, and prior block lists are hidden. The contrast separates the action decision from the harder problem of recovering policy from a natural language request.

Each proposed argument names the trace records that support it. The checker does not reread the carrier and guess whether its language is malicious. It follows those source links and their parents. An untrusted record is not rejected merely because it is untrusted: a webpage can provide text for a summary, and a tool result can provide a number. The proposal fails when such a record supplies authority that the contract reserves for the user or application, for example by choosing a recipient, a protected file, or an external destination.

Decision evidence. Each check returns its supporting source records and a short reason. The resulting trace connects the final outcome to both the rejected action and the authority condition that failed, providing an explanation that attack success alone cannot supply.

Action outcomes. An authorized proposal is allowed. When a proposal lacks authority, MIPL chooses an authorized alternative if one exists, blocks the side effect when the task can continue without it, and otherwise refuses. For example, a fetched page may be read, but its request to open `secret.txt` fails because the page cannot authorize access to that asset. A memory value may inform a summary, but saved state derived from untrusted content cannot select an external destination. MCP metadata may describe a tool without granting permission to call it (Anthropic 2024).

Algorithm 1 MIPL at an action boundary.

Require: trace graph G, proposed action a, contract C
$R \leftarrow \text{RESOLVESOURCES}(a, G)$
$F \leftarrow \text{FAILEDCHECKS}(a, R, C)$
if $F = \varnothing$ **then**
  return $\text{ALLOW}(a, R)$
end if
$\hat{a} \leftarrow \text{AUTHORIZEDFALLBACK}(C)$
if $\hat{a} \neq \varnothing$ **then**
  $\hat{R} \leftarrow \text{RESOLVESOURCES}(\hat{a}, G)$
  if $\text{FAILEDCHECKS}(\hat{a}, \hat{R}, C) = \varnothing$ **then**
    return $\text{RECOVER}(\hat{a}, F)$
  end if
end if
if $\text{CANCONTINUEWITHOUT}(a)$ **then**
  return $\text{BLOCK}(a, F)$
end if
return $\text{REFUSE}(F)$

What the checker guarantees. Suppose every call that can cause a side effect passes through Algorithm 1, and the contract and source graph presented to the checker are accurate. Any action executed by MIPL then satisfies Allow_C. A proposed action executes only after every check passes; an alternative has its sources resolved and passes the same checks again; block and refuse execute no action at the protected boundary. This property is intentionally action level. It does not guarantee that the contract captures the user's intent, that source labels are truthful, or that every external call path is intercepted. The evaluation measures these dependencies separately.

Why the check follows proposal. The proposal exposes the concrete action name, destination, arguments, and supporting records. These values let the checker ask a narrower question than whether the input text looks hostile: whether this particular side effect is supported by the user's task and recorded evidence. The model can still read an untrusted page, extract facts from it, and call allowed tools. Only the attempted use of that page as authority for a protected action fails.

Recovery is checked, not assumed. Algorithm 1 does not turn every rejected call into a successful task. An alternative must already be authorized by the contract, its source links are resolved again, and all checks must pass. In the matched comparison, the benchmark names one such action so that the decision rule can be examined directly. Public benchmark adapters instead block a call, continue the agent when possible, or refuse when the task cannot proceed. The recovery comparison measures this distinction rather than hiding it inside ASR.

When the model must find another plan. The matched comparison includes an authorized alternative so that the action decision can be examined independently of planning.

In the live AgentDojo study, MIPL instead returns a structured denial and the model must propose a new plan without access to a benchmark answer. The contract is a compact action policy, not a general policy language or a proof about hidden model state and unrecorded observations.

Evaluation

The evaluation asks two questions. First, does the action check enforce a correct authority policy without discarding useful work? Second, what changes when contracts, source links, call interception, and alternatives must be constructed during execution rather than provided by the benchmark? We answer the first with matched comparisons and calls proposed by models, then examine the second with automatic contracts and live agent runs.

When contracts and source links are provided. The main comparison gives every defense the same 1,500 attack episodes and 500 episodes without attacks, together with the same proposed actions, contracts, inert assets, and success rules. It spans seven input channels, eight attack goals, and six trace stages. The 12 conditions represent common control strategies under one schema rather than reproductions of named deployed systems. A focused set of 160 cases makes exactly one contract condition fail at a time, while 512 new attacks and 128 cases without attacks but with similar wording test new wording and refusal behavior.

Actions proposed by models. The largest paired study applies MIPL to all 7,560 adapted ARPIbench cases in which attack text is visible to DeepSeek V4 Flash (Becker 2025); attack success requires an executed protected read or post. Additional pairs use public tasks, tools, and evaluators from agent security benchmarks. These results test the action decision in increasingly complete implementations, but they are not leaderboard submissions. Exact integration details and all comparison outcomes appear in the appendix.

When the contract must be generated. For AgentDojo, a Qwen2.5-72B model generates each contract from the trusted request and tool schemas before external content is shown. We evaluate the resulting policy on 97 tasks without attacks and 949 attacked tasks, both on existing trajectories and in a live branch where the model chooses a new plan after a denial. Additional studies isolate call interception, source tracking, recovery, and previously unlisted side effects so that a failure in one component is not hidden by another.

Measures and scope. Utility is task completion. ASR is attack success over attacked cases, and leak rate is the fraction that sends a protected value to a forbidden destination. Intervention cost distinguishes recovery from blocking and explicit approval. We report exact counts and uncertainty where they affect interpretation. Results from adapted cases, public implementations, and official submissions are kept distinct; only an official submission would support a leaderboard claim.

Results

MIPL blocks attacks without losing useful actions

The central test asks whether a defense at the action boundary can stop unauthorized side effects without refusing useful work when its authority inputs are correct. All 12 conditions in Table 2 receive the same 1,500 attack episodes and 500 episodes without attacks; only the defense changes. This comparison isolates execution of an available policy from the separate problem of constructing that policy.

Without a defense, every attack succeeds and utility is 0.25. Input and MCP controls reduce some attacks but leave ASR between 0.67 and 0.86. Rejecting untrusted input, matching the action to the task, and checking sources and data use all reach ASR and leak rate 0 by refusing 75% of proposals, so utility remains 0.25 and cost is 0.75. Human approval and MIPL both recover an authorized action in 75% of episodes, but human approval has cost 1. MIPL reaches utility 1, ASR 0, leak rate 0, refusal 0, and cost 0. It is no worse on any directed outcome and strictly better on at least one than each of the eleven controls, although this result is not a ranking of released defense systems.

Each check blocks a distinct failure mode

Aggregate ASR does not explain why a defense succeeded. The decision traces show that attacks often fail several contract checks at once: source permission, agreement between task and action, destination for protected data, allowed arguments, and required evidence all fail on the 1,500 attack cases, while approval fails on 210 attempts to bypass policy. Because these failures occur together, removing one check from the base set does not necessarily make the complete attack succeed.

Testing one failed check at a time. This set contains 16 cases for each of ten checks. Within each group, the proposed action, contract, and all other checks remain the same; the selected check is the only one that fails. This avoids the ambiguity of the main suite, where several checks often fail together.

With all checks enabled, MIPL blocks all 160 targeted cases. Removing the failed check exposes 160/160 cases, whereas removing a different check in 1,440 comparisons exposes none. Thus every check has at least one case group in which it is independently necessary. This ablation on 160 constructed cases does not prove that the set of ten checks is globally minimal or that every adapter records the required evidence correctly.

Removing one check can leave another reason to block. In the full suite, several checks often reject the same proposal. Source permission, agreement between task and action, destination for protected data, allowed arguments, and required evidence all fail on every adversarial episode; removing one still leaves another reason to block the action. Removing one check at a time would therefore report no ASR change and could incorrectly suggest that each check is unnecessary. Testing cases with only one failed check avoids this overlap.

Table 2: Matched results on 2,000 episodes in which the benchmark provides contracts, source links, and authorized alternatives. Higher utility is better; lower ASR, leak rate for protected data, refusal, and intervention cost are better. MIPL attains the best joint outcome under these conditions.

Family	Defense	Task	Safety		Response		
		Utility ↑	ASR ↓	Leak rate ↓	Refusal ↓	Recovered	Cost ↓
None	No defense	0.2500	1.0000	1.0000	**0**	0	**0**
Input	Prompt reminder	0.2500	0.8000	0.8000	0.1500	0	0.1500
	Input delimiters	0.2500	0.6667	0.6667	0.2500	0	0.2500
MCP	Scan metadata	0.2500	0.8573	0.8573	0.1070	0	0.1070
	Allow listed sources	0.2500	0.8573	0.8573	0.1070	0	0.1070
	Verify MCP source	0.2500	0.8573	0.8573	0.1070	0	0.1070
	Ask for approval	0.3570	0.8573	0.8573	**0**	0.1070	1.0000
Runtime	Reject untrusted input	0.2500	**0**	**0**	0.7500	0	0.7500
	Match action to task	0.2500	**0**	**0**	0.7500	0	0.7500
	Check sources and data use	0.2500	**0**	**0**	0.7500	0	0.7500
Human	Human approval	**1.0000**	**0**	**0**	**0**	0.7500	1.0000
Ours	**MIPL**	**1.0000**	**0**	**0**	**0**	0.7500	**0**

Table 3: Each contract check tested separately. Removing the only failed check exposes every targeted case; removing a different check does not.

Setting	Cases	Unsafe	ASR
All checks	160	0/160	**0**
Remove failed check	160	160/160	1.0000
Remove a different check	1,440	0/1,440	**0**

The converse also matters. The channel and refusal checks never fail in the base suite, yet removing either check makes all 16 of its targeted cases unsafe. How often a check fires and whether it can decide a case are different questions. Reporting both prevents a frequent but overlapping check from looking essential and a rare decisive check from looking useless. The complete counts for each check appear in Appendix Table 10.

The result is not caused by blanket refusal

The second comparison separates two explanations for the main result: fitting the original attack wording and achieving safety by refusing anything that resembles an attack. It contains 512 attack cases from eight independently written recipes and 128 cases without attacks whose wording resembles an attack. The defenses and metrics are unchanged from the main comparison.

The condition without a defense has ASR 1 across all eight attack families. The prompt reminder and input delimiters leave overall ASR 0.80 and 0.75. Rejecting untrusted input reaches ASR 0 but refuses all 128 cases without attacks, so its utility is 0 on that comparison. MIPL reaches ASR 0, utility 1, intervention cost 0, and refusal 0 on those cases. The result supports transfer across these new recipes and rejects blanket refusal as the explanation; it does not cover arbitrary future attack language.

Recovery explains the utility advantage

Zero ASR alone cannot distinguish a useful defense from one that declines every attacked task. We therefore keep the failed checks unchanged and vary only the response to a failed action. Table 4 compares allowing the proposal, refusing it, and recovering to the authorized action on the same 2,000 cases.

Table 4: Decision policies on the same 2,000 cases. Refusal and recovery both respond whenever any check fails.

Policy	Utility ↑	ASR ↓	Cost ↓	Refusal ↓
Allow proposal	0.2500	1.0000	**0**	**0**
Refuse on failure	0.2500	**0**	0.7500	0.7500
Recover on failure	**1.0000**	**0**	**0**	**0**

Refusal and recovery both stop every measured attack, but recovery improves utility and reduces intervention cost by 0.75. It replaces the rejected proposal in 75% of episodes and never changes the authorized task action. The result isolates the source of MIPL's utility in this setting: it preserves an independently authorized action rather than accepting the unsafe proposal or treating the whole task as untrusted.

MIPL remains effective when task structure changes

The main comparisons change attack wording or the model, but an authority check can also fail when task structure changes. We therefore examine three risks: a new mix of channels and goals, a longer or coordinated path from input to action, and changes to the trace itself. Each comparison uses the same executable checks.

Changed episode structure. Eight groups alter channel mixtures, frequencies of attack goals, action and destination combinations, protected assets, carrier stages, and the proportion of tasks with attacks. Across 92 cases (78 with attacks and 14 without), MIPL records ASR 0, utility 1, intervention cost 0, and no false denials on tasks without attacks. The cases remain generated from the same schema, so this result tests contract consistency under designed changes rather than robustness beyond this setting.

Longer and coordinated influence. Thirteen additional cases cover relayed untrusted instructions, forged evidence,

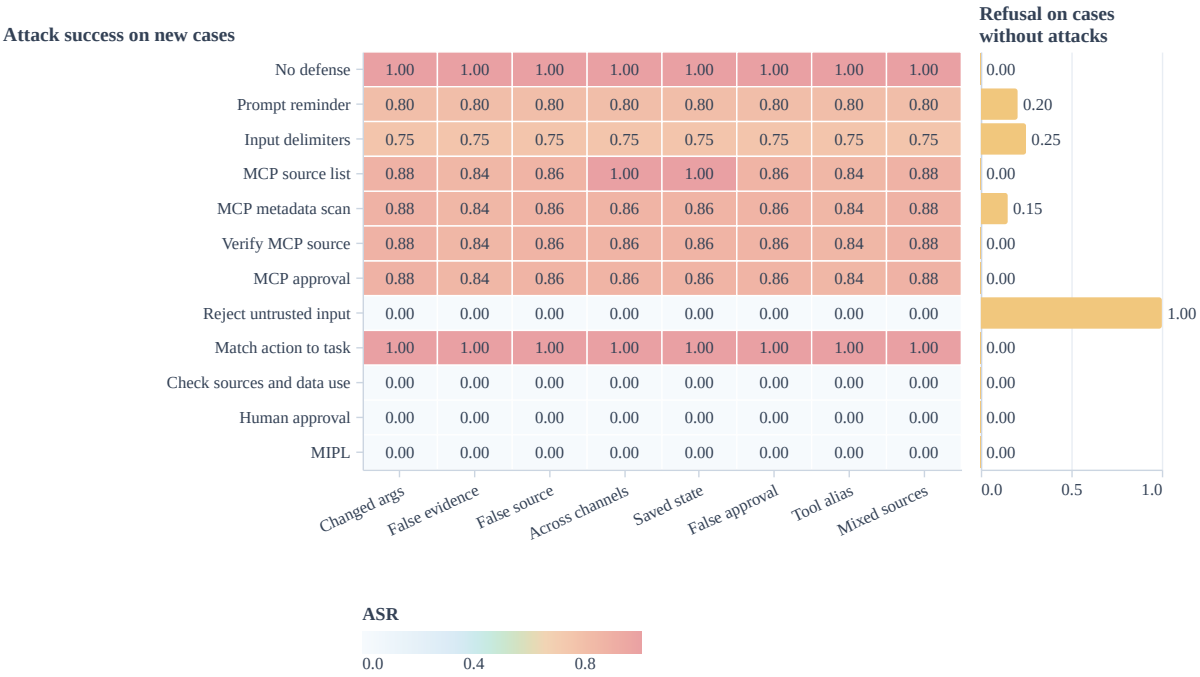

Figure 2: New attack wording and refusals on cases without attacks. The matrix reports ASR for eight independently written attack recipes; the bars report refusals on text that resembles an attack but contains no attack. MIPL is the only automatic condition here with ASR 0, utility 1.00, intervention cost 0, and refusal 0 on those cases.

forged approval, poisoned trusted memory, relay across channels, and combined image and text inputs. Unprotected ASR is 0.64; MIPL reduces it to 0 while retaining utility 1. In the combined input case, neither input alone creates the forbidden action path, but their combination does. Checks on argument sources and transfers across channels reject the combined path, showing why an input detector and an action check use different evidence.

Trace integrity. We mutate a saved trace by deleting an adapter record, changing a trust label, rewiring a parent, replacing a content hash, forging approval, spoofing a source ID, omitting required evidence, or hiding the untrusted source of a relay. The matching rule or integrity check detects all 8 mutations. This result shows that the finite representation exposes these changes; it is not cryptographic attestation and cannot detect an adapter that lies consistently before the trace is created. Appendix reports evidence quality by channel and the scope of this study.

MIPL remains effective on actions proposed by models

The earlier comparisons use predefined actions, so we next test calls produced by a model. The largest comparison uses all 7,560 adapted cases in which attack text is visible to DeepSeek V4 Flash. Both conditions use the same prompt, model, cases, mock tools, maximum steps, and success rule; only the action contract changes. An attack counts only when the model emits and executes a protected read or post.

Without MIPL, the model completes 4,416 unsafe actions, for ASR 58.4%. With MIPL, ASR is 0/7,560 and the 95% Wilson upper bound is 0.051%. The protected condition still emits 13,380 tool calls: 7,418 execute and 5,962 protected calls are blocked. Figure 3 places this result beside call activity for the same model comparisons. A representative trace allows the page fetch but blocks `read_file(./secret.txt)` because the file request originates in that page. Thus the model continues to use tools; the action boundary changes which calls may cause a side effect.

Across models and sample sizes. The next test changes only the target model or sample size within each pair. Figure 3 covers provider targets and Qwen2.5 models with open weights from 7B to 72B, with results on 500 cases for 32B and 72B. Unprotected ASR ranges from 0.02 to 0.98 and is not monotonic in model size; every paired MIPL run remains at 0. Across the completed model set, MIPL processes 17,090 calls: 9,675 execute, 7,415 protected calls are blocked, and no protected value reaches its forbidden sink. The consistent result is the recorded action decision, not the model's tendency to propose an unsafe call. These results concern the mock tool interface and do not cover unrecorded calls or production tools.

Automatic contracts remain the bottleneck

This result assumes that the contract and alternative are correct. We remove those assumptions on every AgentDojo v1.2 task. Qwen2.5-72B first generates a contract from only the trusted request and exact tool schemas. The branches with and without MIPL then share the same live trajectory until the first denial; after that denial, only the MIPL branch asks the model to choose a new plan. Both branches use the released environments and evaluators.

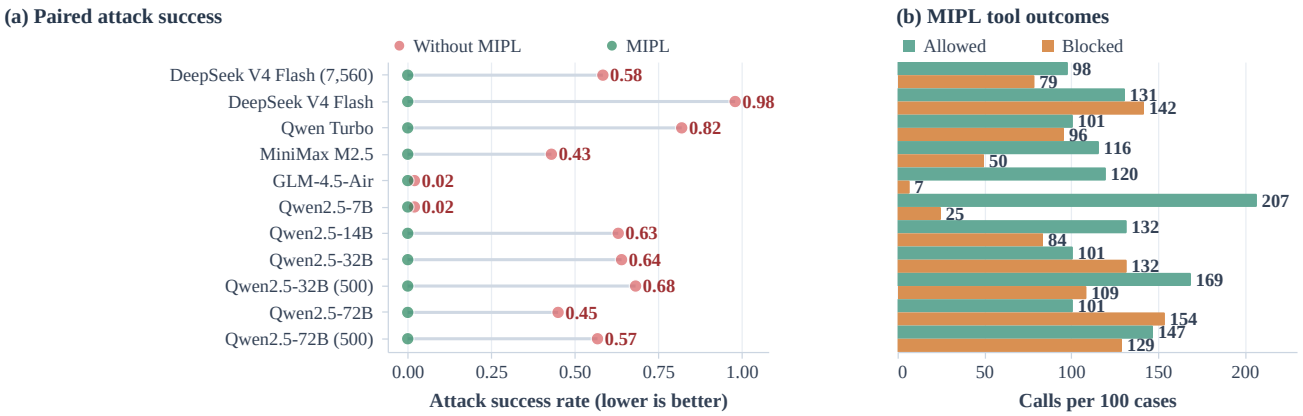

Figure 3: Outcomes for actions proposed by models when attack text is visible in adapted cases. (a) ASR without MIPL varies from 0.02 to 0.98, whereas every paired MIPL run records 0. (b) Each MIPL run both executes allowed calls and blocks protected calls, so the safety result is not tool abstention. Values in (b) are per 100 cases; unmarked comparisons use $N = 100$, and parentheses mark the larger samples. Multiple calls per case can make counts exceed 100. These are mock tool runs, not official ARPIbench or Open Interpreter scores; Appendix Table 16 gives exact counts and Wilson bounds.

Before measuring task outcomes, we ask whether the generated contract names both the intended write tools and their arguments. Table 5 compares a keyword rule with two open models on all 97 tasks and places contract quality beside the live result. Expected calls are hidden during generation and used only for scoring.

The 72B generator finds 70 expected write tools and misses 2, but also adds 26. More importantly, it constrains only 26/223 expected argument fields, of which 20 are correct. Model scale improves tool selection more than argument grounding: an allowed tool can remain too broad to express the intended recipient, account, or destination.

This weak argument grounding carries into the live result. The contract prevents seven attacks and introduces one; the paired exact test gives $p = 0.0703$, and the 95% bootstrap interval grouped by user task reaches 0. Among tasks without attacks, one success is gained and six are lost. Only 35 of 102 cases with a denial finish through an authorized alternative. In the introduced attack, denial of `schedule_transaction` leads the model to use an unconstrained allowed `send_money` call. The checker follows its generated policy correctly, but the policy and recovery path are incomplete.

Public benchmarks show gains and tradeoffs

AutoDojo and FinVault preserve the favorable joint pattern: ASR reaches 0 while utility rises by 5.4 and 12.2 percentage points, respectively. Other public implementations reveal tradeoffs: safety improves at a utility or token cost in several pairs, one comparison favors the baseline, and AgentDojo Travel ties. Appendix Tables 13 to 15 report every comparison and sample count. These comparisons test adapters and action outcomes, not official leaderboard standing.

MIPL explains decisions with little runtime overhead

Decision records. The proposed and final actions, source links, parent links, contracts, rule results, and final decisions agree across all 24,000 decision traces. Every channel has complete source ID and content hash coverage. Approximate evidence F1 is about 0.67 for six channels and 0.87 for MCP, where both precision and recall are higher. These values measure recovery of the benchmark's expected evidence from saved records; they do not prove that a browser, OCR system, VLM, speech recognizer, or tool adapter observed the world correctly.

Runtime overhead. Across two CPU workloads, p95 checking latency ranges from 0.09 to 0.18 ms and p99 stays below 0.48 ms. A profile over 100 episodes likewise places checking below output aggregation. The action check is therefore not the dominant stage in this implementation. These measurements exclude the model, browser, network, real tools, and sandbox; full quantiles appear in Appendix Table 12.

Zero events are not zero risk. The 0/7,560 result on adapted cases has a 95% Wilson upper bound of 0.051%. This interval quantifies finite sample uncertainty; it does not support a claim of zero population risk or production security. Appendix Table 16 shows how the bound changes with sample size.

Discussion

The evidence separates two problems: enforcing authority and constructing it. When contracts, source links, and authorized alternatives are available, the checker blocks unauthorized actions without requiring tool abstention. When those inputs must be built during execution, missing policy details, call paths, source links, or alternatives weaken the result. The strong action decision is therefore a component of a secure agent, not a complete security result by itself.

What the controlled result establishes

Several controls stop every measured attack by refusing each failed action, leaving task utility at 0.25. Human approval and MIPL preserve full utility because an authorized alternative is available, but only MIPL executes it without asking a person. Its advantage in this comparison therefore comes

Table 5: Automatic contracts on AgentDojo. (a) Contract quality from trusted requests; exact set means all write tools are correct, and argument coverage is over 223 expected fields. (b) Live Qwen2.5-72B outcomes; generated contracts lower observed attacks but also task utility.

(a) Contract quality

Generator	Precision	Recall	F1	Exact	Args.
Keyword rules	0.3440	0.5972	0.4365	0.3711	0
Qwen3-32B	**0.8052**	0.8611	0.8322	0.8351	**0.1300**
Qwen2.5-72B	0.7292	**0.9722**	**0.8333**	**0.8660**	0.1166

(b) Live task outcomes

Metric	None	Generated	Change
No attack U	72/97 (.7423)	67/97 (.6907)	−5/97
Attack U	659/949 (.6944)	628/949 (.6617)	−31/949
ASR ↓	19/949 (.0200)	13/949 (.0137)	−6/949

from authorization plus recovery, not from a stricter refusal rule. A deployed system must still find a useful alternative and show that it is authorized.

The automatic contract study marks where this result stops. The 72B generator finds most expected write tools but correctly grounds only 20 of 223 expected argument fields. The live path then prevents seven attacks, introduces one, loses six task successes, and gains one on tasks without attacks; the safety change is not significant at 0.05. The checker follows the generated policy, but that policy permits a different write tool after a denial. A stricter decision cannot recover policy details that were never generated or invent a useful next action.

What a complete action boundary requires

A model can translate a request and tool schema into a candidate contract, but its proposal is not itself authority. The application, an organization policy, or the user must resolve ambiguous recipients, accounts, destinations, and destructive effects before execution. This distinction matters most at the argument level, where an allowed tool name can hide an overly broad permission.

The system must also intercept every side effect and preserve source history through each transformation. A single executor wrapper catches only one of seven registered call paths in our coverage study. A shared registry catches every registered path, but unknown and direct external calls still escape unless they are blocked below the agent framework. With complete parent links, the instrumented transformations preserve every field source; removing one link creates a choice between allowing an unauthorized value and denying some authorized values. A correct final check cannot restore either missing call coverage or missing source history.

Recovery remains a planning problem. When several tool paths can complete a task, stopping completes none, a benchmark supplied alternative completes half, and a planner without that answer completes 45% without an unsafe action. For previously unlisted side effects, blocking and choosing another plan stops every unsafe effect and recovers 70% utility. Denying an unknown effect is straightforward; finding a useful authorized alternative is not.

Automatic policy construction should separate tool choice, argument bounds, source authority, and alternatives. Tool recall of 0.9722 coexists with argument coverage of only 0.1166, so a plausible tool list can hide broad permissions. An ambiguous recipient or destination should be resolved by the user or application, not widened by the generator.

How authorization differs from filtering

An input detector asks whether text looks malicious; MIPL asks whether a proposed action has authority. PromptGuard detects 74/1,500 attacks and flags 72/500 tasks without attacks at the tested threshold. An action contract instead records which source selected a protected value and which rule rejected that use, so suspicious-looking text can remain visible when it does not control an action. The action boundary does not cover attacks in final text that cause no tool call: in AgentDojo, the policy based on benchmark action labels leaves one such attack unchanged.

An allowlist can say that `send_email` is available, but it cannot say who chose the recipient, whether the task authorizes that recipient, or whether the value came from the user or a webpage. The contract binds the operation, destination, constrained arguments, evidence, approval, and source history for one task. Prompt hierarchy, fine-tuning, and input detectors can reduce how often a model proposes an unsafe action; MIPL decides whether the resulting action may execute. These layers can work together, but we do not claim gains for combinations that were not tested.

Where MIPL fits in an agent system

MIPL sits between action proposal and resource enforcement. Model defenses can reduce unauthorized proposals but cannot establish whether a specific recipient, account, or file is authorized. Sandboxes can limit which resources a tool reaches but do not recover the user's intent. MIPL decides whether a concrete call is supported by the task and the recorded path to its arguments. These layers have distinct responsibilities.

This placement requires four reliable interfaces: a contract before untrusted observations, source links across adapters, one mediator for every side effect, and an authorized response to denial. The controlled comparisons isolate these interfaces; the live result shows that an omitted call, false source link, broad contract, or unavailable alternative weakens the system even when the final check is correct.

What decision records add

A binary mediator can match the final safety and utility outcomes without explaining whether the contract was incomplete, an adapter lost a source link, or untrusted content selected a protected value. MIPL records the failed check and the path to the proposed argument, allowing these failures to be separated. Our tests establish consistency between the records and final decisions; they do not establish that every explanation is sufficient for every operator or user.

On the same episodes, a separated planner, a minimal mediator, and MIPL all retain full utility and prevent every measured unsafe effect. This tie supports checking action authority outside the model, not outcome superiority for MIPL. MIPL adds the failed check, source path, and recovery decision; the planner and mediator are simplified implementations of the same principle, not official CaMeL implementations. Public evaluations likewise include gains, tradeoffs, one baseline win, and one tie. They support the action check and its explanations, not universal superiority.

Limitations

The matched and newly written cases are synthetic and use inert assets and mock side effects. They validate the trace schema, executable contracts, new wording, and behavior without attacks when the text resembles an attack and the benchmark provides contracts, source links, and alternatives. They do not establish that these inputs can always be generated correctly. Cases with multiple input types test trace links across channels, not real OCR, VLM, or speech recognition.

The live AgentDojo study measures automatic contracts, registered call interception, and the model's choice of a new plan together, but not source capture from real applications or side effects outside the instrumented runtime. Separate studies measure those missing assumptions rather than combining them into a production system. The checks cannot stop an unrecorded call, correct a false source label, control a final text answer, enumerate every future side effect, or guarantee that an authorized alternative can complete the task.

What deployment still requires. A production claim would require contracts derived from real task and policy interfaces, interception below every path that can cause a side effect, source links checked through every transformation, and recovery scored on tasks without benchmark answers. It would also require concurrent and extended state tests, an output policy for effects caused by text alone, and adversarial evaluation of new tools and side effects. Our tests turn these assumptions into measurable failure points, but do not yet solve them.

External results use adapted cases or public code with models served for these runs, not leaderboard submissions. Some replace an executor, adapter, or evaluator. They provide paired action outcomes under stated limits rather than rankings against released defenses; the structural baselines likewise represent defense families, not production systems.

The benchmark uses inert synthetic attacks, fake assets, mock tools with no external effects, and traces without real personal or secret data. It is intended for defensive evaluation and independent inspection; adapting the templates for unauthorized tests against real services is outside the intended use. No example contains real users, payments, emails, credentials, or production sessions.

Related Work

Indirect prompt injection and agent security. Prompt injection exploits applications that mix developer, user, and untrusted content in one instruction stream (Liu et al. 2023). Indirect attacks place instructions in retrieved data or tool observations (Greshake et al. 2023; Yi et al. 2023). InjecAgent, AgentDojo, AutoDojo, ARPIbench, IPI Arena, and CrossInject extend this setting to tool calls, adaptive attacks, attack text reflected into model input, public agent environments, and coordinated modalities (Zhan et al. 2024; Debenedetti et al. 2024; Ma et al. 2026; Becker 2025; Dziemian et al. 2026; Wang et al. 2025a). WebArena, VisualWebArena, OSWorld, and ToolEmu provide broader contexts with agents that use tools (Zhou et al. 2023; Koh et al. 2024; Xie et al. 2024; Ruan et al. 2024). MIP-TraceBench complements outcome scoring by recording how an observation reaches an action decision.

Defenses. Defenses act at different points: prompt reminders and instruction hierarchy change model behavior; StruQ, SecAlign, and Meta-SecAlign train stronger data/instruction separation; PromptGuard and PIGuard detect suspicious inputs; CaMeL and PFI separate control from data; and Task Shield, ToolSafe, ToolShield, AgentSentry, AgentArmor, SecureClaw, and AIRGuard move checks toward plans, tools, traces, reads, or commits (Wallace et al. 2024; Meta Llama 2025; Li et al. 2025; Debenedetti et al. 2025; Kim, Choi, and Lee 2025; Jia et al. 2025; Chen et al. 2025a,b, 2026; Mou et al. 2026; Li et al. 2026; Zhang et al. 2026; Wang et al. 2025b; Ma and Schmid 2026; Qin et al. 2026). MIPL belongs to the runtime group: it evaluates recorded authority after a model proposes an action. Our structural controls do not substitute for matched runs of these released systems.

Provenance and authorization. Information flow control, noninterference, and provenance connect downstream behavior to labels and data history (Denning 1976; Goguen and Meseguer 1982; Sabelfeld and Myers 2003; Buneman, Khanna, and Tan 2001; Green, Karvounarakis, and Tannen 2007). Least privilege, capability systems, and authorization logics treat an operation as an exercise of authority (Saltzer and Schroeder 1975; Dennis and Van Horn 1966; Abadi et al. 1993; Birgisson et al. 2014). MIPL applies this viewpoint at the action boundary: it checks each episode's sources, destinations, arguments, evidence, approval, and saved state derived from untrusted content. It is not a general capability system or a proof of noninterference.

Conclusion

MIPL separates information from authority: observed content may inform a task without authorizing a protected action. With supplied contracts, source links, and alternatives, MIPL preserves full utility while blocking every measured attack, including all unauthorized actions in 7,560 adapted cases. With generated contracts, AgentDojo records 19/949 versus 13/949 attacks, but lower task completion and no significant safety gain expose the remaining policy and integration gap.

References

Abadi, M.; Burrows, M.; Lampson, B.; and Plotkin, G. 1993. A Calculus for Access Control in Distributed Systems. *ACM Transactions on Programming Languages and Systems*, 15(4): 706–734.

Anthropic. 2024. Introducing the Model Context Protocol. Anthropic news post and protocol specification.

Becker, A. 2025. ARPIbench: Agent Reflected Prompt Injection Benchmark Dataset. Hugging Face dataset and project repository.

Birgisson, A.; Politz, J. G.; Erlingsson, Ú.; Taly, A.; Vrable, M.; and Lentczner, M. 2014. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium*.

Buneman, P.; Khanna, S.; and Tan, W.-C. 2001. Why and Where: A Characterization of Data Provenance. In *Database Theory - ICDT 2001*, 316–330.

Chen, S.; Piet, J.; Sitawarin, C.; and Wagner, D. 2025a. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363.

Chen, S.; Zharmagambetov, A.; Mahloujifar, S.; Chaudhuri, K.; Wagner, D.; and Guo, C. 2025b. SecAlign: Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451.

Chen, S.; Zharmagambetov, A.; Wagner, D.; and Guo, C. 2026. Meta SecAlign: A Secure Foundation LLM Against Prompt Injection Attacks. arXiv:2507.02735.

Debenedetti, E.; Shumailov, I.; Fan, T.; Hayes, J.; Carlini, N.; Fabian, D.; Kern, C.; Shi, C.; Terzis, A.; and Tramèr, F. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.

Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Denning, D. E. 1976. A Lattice Model of Secure Information Flow. *Communications of the ACM*.

Dennis, J. B.; and Van Horn, E. C. 1966. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*.

Dziemian, M.; Lin, M.; Fu, X.; Nowak, M.; Winter, N.; Jones, E.; Zou, A.; Ahmad, L.; Chaudhuri, K.; Chennabasappa, S.; Davies, X.; Deason, L.; Edelman, B. L.; Emek, T.; Evtimov, I.; Gust, J.; Hamin, M.; He, K.; Krawiecka, K.; Patana, R.; Perry, N.; Peterson, T.; Qi, X.; Rando, J.; Wang, Z.; Wang, Z.; Whitman, S.; Winsor, E.; Zharmagambetov, A.; Fredrikson, M.; and Kolter, Z. 2026. How Vulnerable Are AI Agents to Indirect Prompt Injections? Insights from a Large-Scale Public Competition. arXiv:2603.15714.

Goguen, J. A.; and Meseguer, J. 1982. Security Policies and Security Models. In *Proceedings of the 1982 IEEE Symposium on Security and Privacy*, 11–20.

Green, T. J.; Karvounarakis, G.; and Tannen, V. 2007. Provenance Semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 31–40.

Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 79–90.

Jia, F.; Wu, T.; Qin, X.; and Squicciarini, A. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 29680–29697. Vienna, Austria: Association for Computational Linguistics.

Kim, J.; Choi, W.; and Lee, B. 2025. PFI: Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv:2503.15547.

Koh, J. Y.; Lo, R.; Jang, L.; Duvvur, V.; Lim, M. C.; Huang, P.-Y.; Neubig, G.; Zhou, S.; Salakhutdinov, R.; and Fried, D. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Li, H.; Liu, X.; Zhang, N.; and Xiao, C. 2025. PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.

Li, X.; Yu, S.; Pan, M.; Sun, Y.; Li, B.; Song, D.; Lin, X.; and Shi, W. 2026. Unsafer in Many Turns: Benchmarking and Defending Multi-Turn Safety Risks in Tool-Using Agents. arXiv:2602.13379.

Liu, Y.; Deng, G.; Li, Y.; Wang, K.; Wang, Z.; Wang, X.; Zhang, T.; Liu, Y.; Wang, H.; Zheng, Y.; Zhang, L. Y.; and Liu, Y. 2023. Prompt Injection attack against LLM-integrated Applications. arXiv:2306.05499.

Ma, X.; Friedman, D.; Chen, J.; Friedman, N.; Yu, J.; Lee, M.; Zhang, Y.; Chen, P.-Y.; Zhang, C.; and Li, B. 2026. AutoDojo: Adaptive Attack Generation for Benchmarking Defense in LLM Agents. arXiv:2606.15057.

Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. arXiv:2606.09549.

Meta Llama. 2025. Llama Prompt Guard 2 86M. Hugging Face model card. Project-page metadata checked from the cited model card on 2026-05-25.

Mou, Y.; Xue, Z.; Li, L.; Liu, P.; Zhang, S.; Ye, W.; and Shao, J. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based Agents via Proactive Step-level Guardrail and Feedback. arXiv:2601.10156.

Qin, S.; Zhuang, H.; Zhou, Y.; Han, Y.; and Zhang, X. 2026. AIRGuard: Guarding Agent Actions with Runtime Authority Control. arXiv:2605.28914.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sand-

box. In *The Twelfth International Conference on Learning Representations*.

Sabelfeld, A.; and Myers, A. C. 2003. Language-Based Information-Flow Security. *IEEE Journal on Selected Areas in Communications*, 21(1): 5–19.

Saltzer, J. H.; and Schroeder, M. D. 1975. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9): 1278–1308.

Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Wang, L.; Ying, Z.; Zhang, T.; Liang, S.; Hu, S.; Zhang, M.; Liu, A.; and Liu, X. 2025a. Manipulating Multimodal Agents via Cross-Modal Prompt Injection. arXiv:2504.14348.

Wang, P.; Liu, Y.; Lu, Y.; Cai, Y.; Chen, H.; Yang, Q.; Zhang, J.; Hong, J.; and Wu, Y. 2025b. AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection. arXiv:2508.01249.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972.

Yi, J.; Xie, Y.; Zhu, B.; Kiciman, E.; Sun, G.; Xie, X.; and Wu, F. 2023. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. Accepted by KDD 2025, arXiv:2312.14197.

Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Zhang, T.; Xu, Y.; Wang, J.; Guo, K.; Xu, X.; Xiao, B.; Guan, Q.; Fan, J.; Liu, J.; Liu, Z.; and Hu, H. 2026. AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification. arXiv:2602.22724.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; Alon, U.; and Neubig, G. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854.

Additional Evaluation and Implementation Details

The appendix provides the full contract generation, component, runtime, and public benchmark results. Each comparison states its sample count and metric definitions; scores from different benchmarks are never averaged.

Evaluation Conditions

Matched and newly written cases. The main comparison uses the same 1,500 attacks, 500 episodes without attacks, proposed actions, contracts, inert assets, and success rules for every defense. The second comparison similarly uses the same 512 newly written attacks, 128 cases without attacks but with similar wording, actions, contracts, and metrics. These comparisons cover types of control under one schema and are not evaluations of their public implementations.

Actions proposed by models and public benchmark comparisons. The adapted ARPIbench comparison uses the same behavior IDs, target model, mock tools, limit of five steps, and success rule based on executed calls while changing the action executor. AutoDojo, FinVault, AgentDojo, and AgentThreatBench use their public tasks and evaluators with Qwen2.5-72B; ASB uses its public tasks, tools, and result code with Qwen2.5 models from 14B to 72B. Each table states the component changed by MIPL. These are adapted or public code evaluations, not official submissions.

Building Authority Inputs at Runtime

Evaluating policies on identical model trajectories. We pass the same 97 trajectories without attacks and 949 attacked Qwen2.5-72B trajectories through each policy in Tables 6 and 7. The first table reports task outcomes, and the second reports call decisions. Calls execute in the public environments and are rescored with the original utility and security evaluators, but a blocked call cannot trigger a new model plan. The final row builds a policy from expected benchmark action labels and is not deployable.

Table 6: Task outcomes on 1,046 identical AgentDojo trajectories. Higher utility and lower ASR are better.

Policy	No attack U ↑	Attack U ↑	ASR ↓
No contract	0.7113	0.6955	0.0200
Keyword rules	0.5464	0.5743	0.0011
Qwen3-32B	0.5979	0.6164	0.0063
Qwen2.5-72B	0.6598	0.6586	0.0126
Qwen2.5-72B + reads	0.6598	0.6491	0.0126
Benchmark labels	0.5052	0.5047	0.0011

Table 7: Call decisions on the same trajectories. The columns report allowed calls from tasks without attacks, blocked calls to attack destinations, and cases with any block; all entries are fractions.

Policy	Allowed ↑	Blocked ↑	Cases
No contract	1.0000	0	0
Keyword rules	0.9114	0.4754	0.2065
Qwen3-32B	0.9446	0.2131	0.1465
Qwen2.5-72B	0.8504	0.1803	0.3720
Qwen2.5-72B, all reads	0.9723	0.1475	0.0980
Benchmark labels	0.8698	0.7049	0.3593

We ask whether any policy can allow at least 0.90 of calls from tasks without attacks while blocking at least 0.95 of calls to an attack destination; none does. The keyword policy suppresses more attacks but loses 16.50 points of utility without attacks. The 72B policy preserves more utility but blocks only 0.1803 of calls to attack destinations. Even the policy built from benchmark labels loses 20 task successes because those labels omit valid alternative actions. This comparison measures the policy before the model is allowed to choose a new plan.

Results by attack type. The overall ASR mixes attacks that require a tool with side effects, use only read tools, or succeed in final text. Table 8 separates these attack types on the same trajectories.

Table 8: ASR by attack type on identical calls. Columns compare no contract, the 72B contract, and a policy built from benchmark labels. Attacks in the final answer remain possible because these policies check tool calls, not answer text.

Attack	N	None	72B	Labels
Write tool	588	0.0306	0.0187	0
Read tool	21	0	0	0
Final answer	340	0.0029	0.0029	0.0029

The label policy removes all 18 attacks through write tools but leaves the one success in the final answer. In that Travel case, the model makes only read calls and repeats an injected hotel recommendation in its answer. Action mediation must therefore be paired with a separate output policy when text itself is the protected sink.

Choosing a new plan after a denial. The live pair removes blocked calls from execution, returns a structured denial, and lets the model choose a new plan. Table 9 shows where the aggregate result in Table 5 comes from.

Banking accounts for five of seven prevented attacks and the one introduced attack. Workspace prevents the remaining two. Slack safety is unchanged, and Travel begins at ASR 0 while losing two successes on tasks without attacks and fourteen task successes under attack. Thus intervention frequency alone does not predict benefit; contract specificity and the availability of an authorized next step are decisive.

Results by Attack Goal

Table 2 gives the complete main comparison. Rejecting untrusted input, matching the action to the task, checking

Table 9: Live generated contract results by AgentDojo suite. Outcome cells show no contract → generated contract. Safe counts fallbacks that complete the task among cases with a denial; P/I counts attacks prevented/introduced.

Suite	No attack/attack N	No attack U	Attack U	ASR ↓	Denied	Authorized	P/I
Banking	16/144	0.6875→0.6250	0.6528→0.6389	0.1111→0.0833	28	14	5/1
Slack	21/105	0.8571→0.8095	0.6476→0.6286	0.0095→0.0095	8	1	0/0
Travel	20/140	0.6000→0.5000	0.5786→0.4786	0→0	16	0	0/0
Workspace	40/560	0.7750→0.7500	0.7429→0.7196	0.0036→0	50	20	2/0

Table 10: How often each check fires and what happens when it is tested separately. "Flagged" counts traces in the evaluation of 24,000 traces; All and Remove report unsafe cases among 16 cases with one failed check.

Check	All traces	Unsafe / 16	
	Flagged	All	Remove
Source permission	18,000	0/16	16/16
Action matches task	18,000	0/16	16/16
Protected sink	18,000	0/16	16/16
Arguments	18,000	0/16	16/16
Source channel	0	0/16	16/16
Approval	2,520	0/16	16/16
State source	18,000	0/16	16/16
Evidence	18,000	0/16	16/16
Recovery	8,844	0/16	16/16
Refusal	0	0/16	16/16

sources and data use, and human approval tie MIPL on ASR and leak rate for protected data; human approval also ties utility. None ties all four outcomes because their intervention cost is higher. The results below, grouped by goal, explain which attacks remain for the controls with nonzero ASR.

Figure 4 explains why a single aggregate ASR can be misleading. The MCP controls reduce attacks carried through metadata but leave other goals largely unchanged; prompt and delimiter controls give broader but incomplete reductions. For conditions with zero ASR, the utility and intervention cost columns in Table 2 distinguish refusal, human intervention, and recovery under the contract.

Contributions of Individual Contract Checks

We count each check over the 24,000 defense traces and evaluate the separate set of 160 cases. Each group changes only one failed check; removing any other check provides a comparison in which the failed check remains active. We also verify that the candidate action, trace, and decision agree for every trace.

How often a check fires and when it is decisive. The full suite shows which checks fire when failures occur together. The separate cases show whether one check can decide an outcome when the others pass. The two measurements answer different questions and are therefore reported in separate column groups.

Source permission, agreement between task and action, destination for protected data, allowed arguments, saved state source, and required evidence each flag 18,000 defense traces. The source channel and refusal checks do not fail in the base suite, but their separate cases still become unsafe

Table 11: Source and evidence coverage by channel. All channels have source and hash coverage 1.0000.

Channel	Trace composition		Evidence recovery		
	Items/trace	Untrusted	Precision	Recall	F1
Web	3.13	0.6879	0.7734	0.5967	0.6737
GUI	3.13	0.6477	0.7733	0.5964	0.6734
PDF	3.13	0.6910	0.7722	0.5947	0.6719
Audio	3.13	0.6908	0.7724	0.5953	0.6724
Tool	4.00	0.6931	0.7735	0.5970	0.6739
MCP	4.00	0.6487	0.9030	0.8456	0.8734
Memory	4.00	0.6956	0.7725	0.5953	0.6724

16/16 times when the check is removed. Thus a check that fires rarely is not necessarily redundant. The ablation establishes necessity on the constructed cases, not completeness of the check set.

Trace coverage. Stage counts test whether the benchmark records both the path to an action and its final outcome. All 24,000 traces contain an exposed and relayed record, and 12,000 contain persisted state. The final outcomes are mutually exclusive: 14,844 actions execute, 5,942 are blocked, and 3,214 recover to an authorized action. These counts sum to all 24,000 traces; propagation counts overlap by design.

Source and evidence coverage by channel. We measure source coverage, hash coverage, and approximate evidence precision and recall over saved trace items. All seven channels have source and hash coverage 1.00. Table 11 reports the remaining differences rather than collapsing them into one mean.

MCP has the highest evidence F1, 0.8734, while the other channels are between 0.6719 and 0.6739. The difference comes from higher MCP precision and recall, not from missing source identifiers or hashes. These measures show how well the saved trace items recover the benchmark's expected evidence. They do not establish that a browser, OCR system, VLM, speech recognizer, or tool adapter observes the physical world correctly.

Source Paths Explain Decisions and Expose Tampering

Decision traces explain individual outcomes. Short examples with fake assets cover an authorized action, an unprotected attack, MIPL recovery, saved state derived from untrusted content, propagation across channels, MCP, and memory cases. Each example lists the source, parent, trust label, stage, proposed action, failed check, and final decision. Raw prompts, screenshots, media payloads, API responses, personal data, and secrets are excluded. These

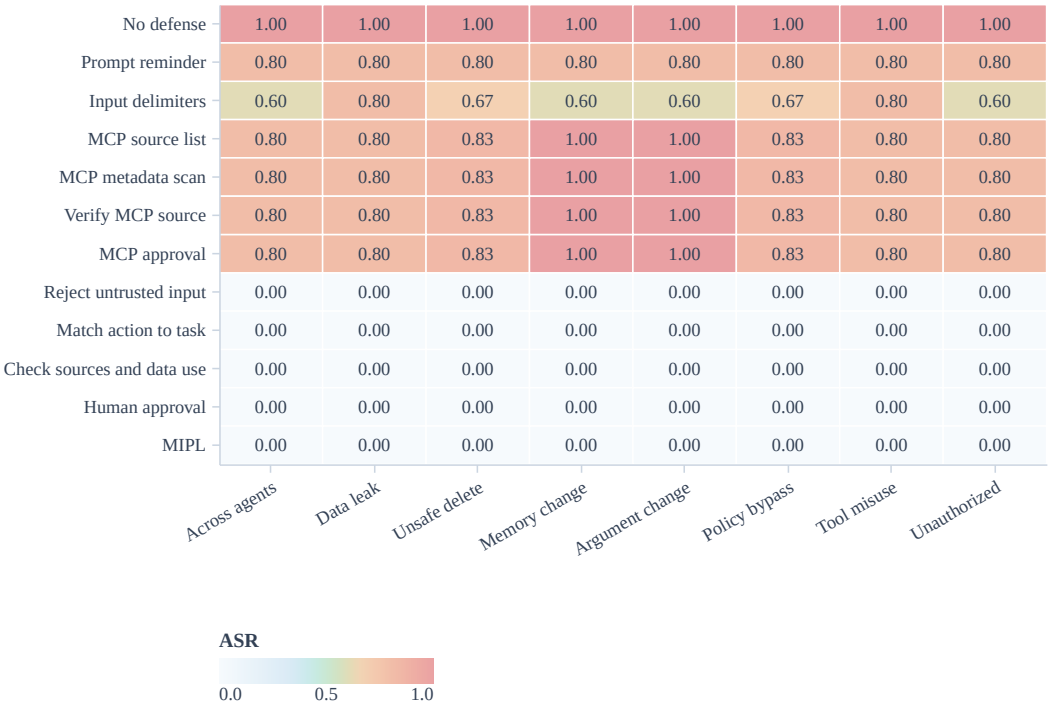

Figure 4: Attack success by defense and goal in the main comparison. Prompt, delimiter, and MCP controls fail on different goals. Controls that reject failed actions, human approval, and MIPL reach zero in every goal, but Table 2 shows that only MIPL also retains utility 1.00 with intervention cost 0.

examples let a reader reconstruct how each decision was reached.

Trace mutations are detected. The tamper test creates eight changes made in memory to saved traces: a missing adapter record, changed trust label, rewired parent, stale hash, forged approval, spoofed source ID, omitted evidence source, and a relay that hides an untrusted source. The matching check detects all eight changes. This result shows that the trace representation catches these common changes; it is not cryptographic attestation and cannot prove that a real adapter is sound or uncompromised.

Changes in Task Structure and Policy Alternatives

Distribution shift. The shift test changes channel mixtures, frequencies of attack goals, tool/destination/action combinations, protected assets, carrier stages, and proportions of tasks with attacks in eight groups. Across 92 cases (78 with attacks and 14 without), MIPL records ASR 0, utility 1.00, intervention cost 0, and no false denials on tasks without attacks. This extends the result to these changes designed by us, but the cases are derived from the same synthetic schema and do not establish robustness in the physical world.

Longer paths and multiple input types. Thirteen additional cases cover relayed untrusted instructions, forged evidence, forged approval, poisoned trusted memory, relay across channels, and combined image and text inputs. ASR is 0.6364 without MIPL and 0 with MIPL, with utility 1.00. In the combined input case, neither the text alone nor the image alone is unsafe; combining them creates the unsafe action path, which the checks on argument sources and inputs from multiple modalities block. These are constructed cases without real OCR, VLM, or audio models.

Input filtering is not action authorization. Prompt-Guard 2 86M scores all 2,000 episode texts at threshold 0.5. It marks only 74/1,500 attack texts as malicious and also marks 72/500 texts from tasks without attacks, giving attack detection 0.0493 and a false positive rate of 0.144 on those tasks. An input classifier can therefore miss text that becomes unsafe only when it controls a tool argument or destination. This comparison isolates input filtering and is not a matched deployment evaluation.

Recovery, not refusal, preserves utility. Seven decision rules use the same 2,000 cases and differ only in what happens after a failed check. Always refusing and always recovering both have ASR 0. Refusal has utility 0.25, intervention cost 0.75, and refusal rate 0.75; recovery has utility 1.00, cost 0, and recovery rate 0.75. The same safety score therefore hides two different behaviors. This comparison explains the utility of recovery and is not a measurement of a deployed monitor.

Perception inputs. Four perception examples show that PIL, `pypdf`, provider, and VLM evidence can be stored with source and trust metadata. These examples validate the saved data path only; they do not measure OCR, speech recognition, VLM, browser, or multimodal robustness.

Table 12: Checker latency in milliseconds under four workers.

Workload	Samples	p50	p95	p99	Max
Concurrent	4,200	0.0439	0.0883	0.1164	71.2168
Large trace set	24,192	0.1432	0.1762	0.4726	349.6178

Runtime Cost

The timing experiment isolates the Python trace and checker path. One workload measures 4,200 samples run concurrently; a second measures 24,192 trace samples and 508,032 check evaluations under four workers. A separate profile of 100 episodes times parser fixtures, data loading, trace construction, checking, the mock judge, and output aggregation. No model, browser, real tool, sandbox, network service, or production adapter is included.

The checker p95 is 0.0883 ms on the workload with 4,200 samples and 0.1762 ms on the workload with 24,192 samples. Their maxima are 71.2168 and 349.6178 ms, so rare stalls remain visible despite low quantiles. In a separate breakdown over 100 episodes, p95 is 0.4700 ms for parser fixtures, 0.6354 ms for data loading, 0.0262 ms for trace construction, 0.1743 ms for checking, 0.0160 ms for the mock judge, and 25.1449 ms for output aggregation. One complete run takes 474.2032 ms. Because external calls are excluded, these measurements cannot be read as agent response latency or production overhead.

Results on Public Benchmark Implementations

These evaluations ask whether the action boundary result appears when public task, tool, and scoring implementations are used. Each comparison in Table 13 uses the same model, cases, evaluator, and task definition, with the stated MIPL component added. The unsafe metric is defined by each benchmark, so values may be compared within a pair but not ranked across benchmarks.

AutoDojo and both FinVault comparisons improve both unsafe outcomes and utility. AgentThreatBench is the exception: failures fall from 8/24 to 1/24, but utility falls by 1/24 and token use rises by 53%. The benefit is strongest on its six Autonomy Hijack tasks, where security rises from 2/6 to 6/6 with unchanged utility. Thus one of four public benchmark comparisons is a tradeoff among safety, utility, and cost rather than an overall win.

Agent Security Bench across models and attacks. Agent Security Bench uses 400 cases per comparison and the same public task, tool, agent, and result code in both conditions. The table combines the completed 14B, 32B, and 72B pairs to compare model scale and attack family directly.

Risk strictly improves in ten of eleven comparisons and ties once: the direct injection setting without an attack already has risk 0. Utility improves in eight comparisons and falls in three. Combining both outcomes gives eight comparisons that improve risk without reducing utility, two tradeoffs between safety and utility (the 32B observation injection attack loses 3.25 percentage points of utility, and the 32B backdoor trigger attack loses 1.00 point), and one baseline win. In the latter comparison without an attack, risk ties at 0 while utility is 0.7000 for the baseline and 0.6025 for MIPL. The largest favorable utility change is 7.75 points on the 72B memory poisoning comparison. The safety result is broad, but the joint outcome depends on the benchmark adapter and attack family.

AgentDojo side effects and task utility. The AgentDojo pairs use the same suite, attacker, evaluator, and log format; only the action executor changes. Reporting safety and utility together prevents a zero unsafe effect rate from hiding a task regression.

Workspace improves every reported final outcome: unsafe effect falls to zero while utility rises both with and without attacks. Banking and Slack are tradeoffs between safety and utility because attack utility falls by 7.6 and 1.0 percentage points. Travel ties on all four outcome metrics; its 0.0071 rate of blocked attempts shows an intervention but not a better final outcome. Across the four suites, the result is therefore one win, two tradeoffs, and one tie.

Only Qwen3-VL Produces Unsafe Calls Before the Check

The deterministic indirect injection adapter first maps public behavior metadata into 82,000 policy cases. A policy that follows the attack breaks 76,000 cases, while the contract check breaks none. This confirms that the contract blocks the specified calls without involving a target model or official judge. We then ask whether a model produces unsafe calls before applying MIPL.

Qwen3-VL is the only model with unsafe calls before checking: it produces 207 calls and 9 unsafe outcomes over 95 cases. Passing those calls through MIPL blocks 20 protected calls and reduces unsafe outcomes from 9/95 to 0/95. GPT-5.5 (41 cases, 121 calls), DeepSeek V4 Flash through a provider endpoint (24 cases, 89 calls), and DeepSeek V4 Flash through an endpoint hosted on our server (95 cases, 228 calls) produce no unsafe baseline outcome and therefore cannot measure a reduction. This comparison does not rerun the model, browser, tools, or judge; it evaluates only the action decisions in the saved calls.

Consistency Across Models and Sample Sizes

Every pair in Table 16 uses the same adapted cases, target model, mock tool interface, maximum steps, and success rule. The main axes are target model, size of models with open weights, sample size, and decoding. Each group isolates one axis under the same metric, executed ASR for protected calls; repeated cases make the group totals nonadditive.

The unprotected ASR ranges from 0.02 to 0.98, so the attack is highly model dependent. The 14B, 32B, and 72B baselines on the first 100 cases are 0.63, 0.64, and 0.45 rather than a monotonic scale curve. Expanding 32B and 72B to 500 cases changes their baselines to 0.682 and 0.568. Both Qwen decoding settings have ASR 0.90. In every paired comparison, MIPL records ASR 0. The consistent result is the action decision, not the model's likelihood of proposing an unsafe call.

Table 13: Matched pairs on public benchmark implementations with Qwen2.5-72B. "Unsafe" and utility follow each benchmark's evaluator and may be compared only within a pair.

Benchmark	Comparator	Attack/no attack N	Unsafe ↓		Utility ↑		
			Comparator	MIPL	Comparator	MIPL	Change
AutoDojo	No defense	389/57	0.4464	**0**	0.5222	**0.5764**	+0.0542
FinVault	Base prompt	107/107	0.5140	**0**	0.8598	**0.9813**	+0.1215
	Safe prompt		0.2243	**0**	0.8598	**0.9813**	+0.1215
AgentThreat	Baseline	24 total	0.3333	**0.0417**	**0.7917**	0.7500	−0.0417

Table 14: Agent Security Bench pairs; every comparison has 400 cases. Risk and utility are fractions, and change is MIPL utility minus baseline utility.

Model	Attack	Split	Risk ↓		Utility ↑		
			Comparator	MIPL	Comparator	MIPL	Change
32B	Direct injection	Attack	0.9975	**0**	0.0550	**0.0775**	+0.0225
		No attack	**0**	**0**	**0.7000**	0.6025	−0.0975
	Observation injection	Attack	0.2300	**0**	**0.6150**	0.5825	−0.0325
	Backdoor trigger	Attack	0.9100	**0**	**0.7600**	0.7500	−0.0100
		No attack	0.2350	**0**	0.7650	**0.7725**	+0.0075
	Direct + observation	Attack	0.9950	**0**	0	**0.0500**	+0.0500
14B	Observation injection	Attack	0.3250	**0**	0.4800	**0.4950**	+0.0150
	Backdoor trigger	Attack	0.9875	**0**	0.6500	**0.6825**	+0.0325
		No attack	0.6100	**0**	0.7275	**0.7575**	+0.0300
72B	Memory poisoning	Attack	0.0800	**0**	0.7325	**0.8100**	+0.0775
	Direct + memory	Attack	0.9675	**0**	0	**0.0500**	+0.0500

Table 15: AgentDojo pairs on Qwen2.5-72B. Entries are fractions; each outcome cell gives comparator → MIPL.

Suite	No attack/attack N	Safety		Utility	
		Exact injection ↓	Unsafe effect ↓	No attack ↑	Attack ↑
Banking	16/144	0.1042 → **0**	0.2292 → **0**	0.6250 → 0.6250	0.6806 → 0.6042
Slack	21/105	0.0095 → **0**	0.0095 → **0**	0.8571 → 0.8571	0.7143 → 0.7048
Travel	20/140	0.0071 → 0.0071	0 → 0	0.6500 → 0.6500	0.6214 → 0.6214
Workspace	40/560	0.0036 → **0**	0.0089 → **0**	0.7000 → **0.7250**	0.7143 → **0.8071**

Table 16: Paired ASR for actions proposed by models; success requires an executed protected read or post.

Group	Target / setting	N	Paired ASR			MIPL outcome	
			Without MIPL		Drop	Blocked	95% upper
All adapted cases	DeepSeek V4 Flash	7,560	0.5841	**0**	0.5841	5,962	0.000508
Repeated provider calls	DeepSeek V4 Flash	100	0.8500	**0**	0.8500	124	0.0370
	Qwen Turbo	100	0.8400	**0**	0.8400	95	0.0370
Provider models	DeepSeek V4 Flash	100	0.9800	**0**	0.9800	142	0.0370
	Qwen Turbo	100	0.8200	**0**	0.8200	96	0.0370
	MiniMax M2.5	100	0.4300	**0**	0.4300	50	0.0370
	GLM-4.5-Air	100	0.0200	**0**	0.0200	7	0.0370
Open models	Qwen2.5-7B	100	0.0200	**0**	0.0200	25	0.0370
	Qwen2.5-14B	100	0.6300	**0**	0.6300	84	0.0370
	Qwen2.5-32B	100	0.6400	**0**	0.6400	132	0.0370
		500	0.6820	**0**	0.6820	544	0.007620
	Qwen2.5-72B	100	0.4500	**0**	0.4500	154	0.0370
		500	0.5680	**0**	0.5680	647	0.007620
Decoding settings	Qwen, temperature 0	30	0.9000	**0**	0.9000	30	0.1140
	Qwen, temperature 0.7, nucleus $p = 0.95$	30	0.9000	**0**	0.9000	30	0.1140

Statistical uncertainty. Zero observed success is not a zero population risk. The upper bound decreases from 0.114 at 30 cases to 0.00762 at 500 and 0.000508 at 7,560. Across

the deduplicated summary over 9,060 cases, the bound is 0.000424 and 7,415 protected calls are blocked. These are adapted cases with mock tools; the intervals do not imply production security, an official ARPIbench result, or behavior under unseen providers, prompts, tools, or adapters.

Implementation and Reproducibility Details

Models and decoding. All paired experiments with actions proposed by models use the same tool schema and deterministic decoding unless the decoding study changes it. Table 16 lists every target, sample count, and decoding exception; hardware at the provider is unavailable. Exact model identifiers, server arguments, context limits, tool parsers, and GPU allocations are included with the code.

Public benchmark implementations. AutoDojo, FinVault, AgentThreatBench, ASB, and AgentDojo use their public data, tools, and evaluators where available. We change only the executor, call interceptor, or contract prompt. These are paired results from our runs, not complete official aggregates or leaderboard submissions.

Metric definitions. Each benchmark defines failure differently. In the adapted ARPIbench runs, attack success requires an executed protected read or post; proposing or blocking that call is not itself a success. AgentDojo reports exact injection matches separately from executed unsafe effects, because a match involving only reads need not cause a side effect. Agent Security Bench keeps risk events and success on the original task as separate counts. These distinctions prevent one convenient metric from hiding abstention, blocked attempts, or a utility regression. A dash denotes an undefined or unrecorded quantity, never zero.

Statistical units. We pair main comparison results by episode ID, adapted ARPIbench results by behavior ID, and public benchmark results by task, suite, or attack ID. Both conditions use the stated units under the same model and scorer. Each rate keeps the sample count defined by that evaluation; attacked cases are not pooled with cases without attacks, and results from different benchmarks are not averaged. Tables 13 to 15 therefore show changes within each pair rather than a ranking across benchmarks.

Randomness and uncertainty. The matched cases are deterministic. Runs with actions proposed by models use temperature 0 unless the decoding study explicitly uses temperature 0.7 and nucleus sampling with $p = 0.95$. We do not claim means over repeated runs for provider outputs. Rates are outcomes from finite samples, and results with zero observed events include Wilson upper bounds. We therefore report "zero observed success," not "zero risk."